Exp No: 1	**Three-Layer Neural Network from Scratch using Backpropagation for Handwritten Digit Recognition**

AIM:

To design and implement a three-layer neural network from scratch in Python (NumPy), train it using the backpropagation algorithm with suitable activation and loss functions, and apply it to recognize handwritten digits.

ALGORITHM / PROCEDURE:

1. Import NumPy and load the handwritten digits dataset (`load_digits`, 1797 images of 8x8 pixels, classes 0-9).
2. Scale the pixel values to the range 0-1, split the data into 80% training and 20% testing sets, and one-hot encode the labels.
3. Build the network: input layer (64 neurons), hidden layer (32 neurons, *sigmoid*) and output layer (10 neurons, *softmax*). Initialise weights randomly and biases to zero.
4. **Forward pass:** `a1 = sigmoid(X.W1 + b1)` and `a2 = softmax(a1.W2 + b2)`.
5. Compute the *cross-entropy* loss between the predicted probabilities and the true labels.
6. **Backward pass:** output error `d2 = a2 - y`, hidden error `d1 = (d2 . W2.T) * a1 * (1 - a1)`, then compute the gradients dW1, db1, dW2, db2.
7. Update the weights using mini-batch gradient descent (learning rate 0.5, batch size 32) and repeat for 200 epochs.
8. Evaluate the trained network on the test set and report the loss, accuracy and sample predictions.

Fig 1: Architecture of the three-layer neural network

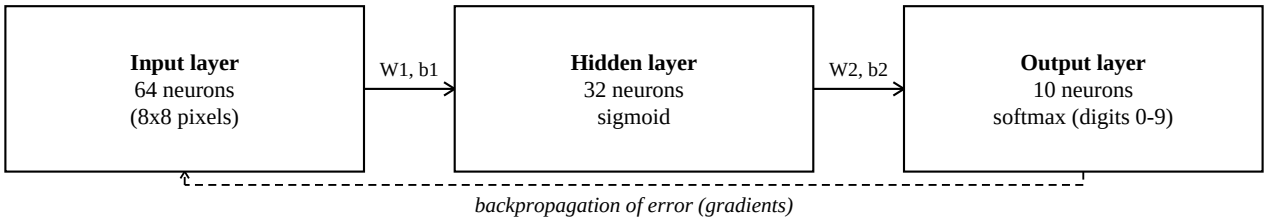
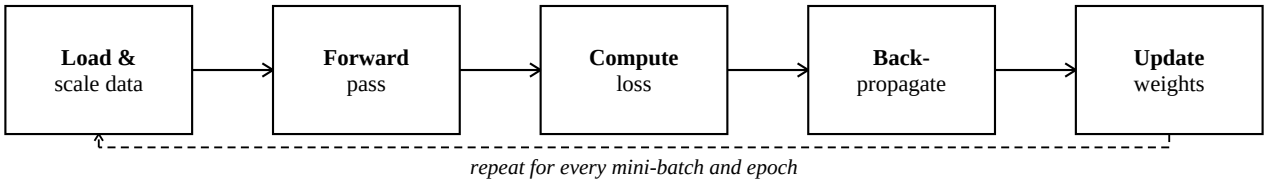


Fig 2: Training loop of the network



Commands / Settings used:

Component	Choice	Details
Dataset	scikit-learn digits	1797 samples, 64 features, 10 classes
Train / test split	80% / 20%	1437 training, 360 testing (stratified)
Hidden activation	Sigmoid	$\sigma(z) = 1 / (1 + e^{-z})$
Output activation	Softmax	probabilities over 10 classes
Loss function	Cross-entropy	-mean(sum(y log p))
Optimiser	Mini-batch GD	learning rate 0.5, batch 32, 200 epochs

PROGRAM:

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

np.random.seed(42)

# ---------- Load handwritten-digit data ----------
data = load_digits()                      # 8x8 handwritten digits, 10 classes
X = data.data / 16.0                      # scale pixels to 0-1
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
Y_train = np.eye(10)[y_train]             # one-hot labels

# ---------- Network: 64 -> 32 -> 10 (input, hidden, output) ----------
n_in, n_hid, n_out = 64, 32, 10
W1 = np.random.randn(n_in, n_hid) * np.sqrt(1.0 / n_in)
b1 = np.zeros((1, n_hid))
W2 = np.random.randn(n_hid, n_out) * np.sqrt(1.0 / n_hid)
b2 = np.zeros((1, n_out))

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def softmax(z):
    e = np.exp(z - z.max(axis=1, keepdims=True))
    return e / e.sum(axis=1, keepdims=True)

def cross_entropy(P, Y):
    return -np.mean(np.sum(Y * np.log(P + 1e-12), axis=1))

def forward(X):
    z1 = X @ W1 + b1
    a1 = sigmoid(z1)                      # hidden layer
    z2 = a1 @ W2 + b2
    a2 = softmax(z2)                      # output layer
    return a1, a2

# ---------- Training with backpropagation ----------
lr, epochs, batch = 0.5, 200, 32
for epoch in range(1, epochs + 1):
    idx = np.random.permutation(len(X_train))
    for s in range(0, len(idx), batch):
        b = idx[s:s + batch]
        xb, yb = X_train[b], Y_train[b]
        a1, a2 = forward(xb)
        # backward pass
        d2 = (a2 - yb) / len(b)           # softmax + cross-entropy gradient
        dW2 = a1.T @ d2
        db2 = d2.sum(axis=0, keepdims=True)
        d1 = (d2 @ W2.T) * a1 * (1 - a1)  # sigmoid derivative
        dW1 = xb.T @ d1
```

```
        db1 = d1.sum(axis=0, keepdims=True)
        # weight update
        W2 -= lr * dW2; b2 -= lr * db2
        W1 -= lr * dW1; b1 -= lr * db1
    if epoch % 20 == 0 or epoch == 1:
        _, P = forward(X_train)
        loss = cross_entropy(P, Y_train)
        acc = np.mean(P.argmax(1) == y_train)
        print(f"Epoch {epoch:3d}/{epochs}  loss: {loss:.4f}  train accuracy: {acc*100:.2f}%")

# ---------- Evaluation ----------
_, P = forward(X_test)
pred = P.argmax(1)
print(f"\nTest loss     : {cross_entropy(P, np.eye(10)[y_test]):.4f}")
print(f"Test accuracy : {np.mean(pred == y_test)*100:.2f}%")
print("\nSample predictions (predicted / actual):")
print(" ".join(f"{p}/{a}" for p, a in zip(pred[:10], y_test[:10])))
```

OUTPUT:

Terminal - Three-layer neural network (backpropagation)

```
root@ip-10-10-5-10:~# python3 exp1.py
Epoch   1/200  loss: 1.8152  train accuracy: 51.15%
Epoch  20/200  loss: 0.1127  train accuracy: 97.70%
Epoch  40/200  loss: 0.0579  train accuracy: 98.89%
Epoch  60/200  loss: 0.0393  train accuracy: 99.51%
Epoch  80/200  loss: 0.0267  train accuracy: 99.86%
Epoch 100/200  loss: 0.0197  train accuracy: 99.86%
Epoch 120/200  loss: 0.0154  train accuracy: 99.93%
Epoch 140/200  loss: 0.0126  train accuracy: 99.93%
Epoch 160/200  loss: 0.0103  train accuracy: 100.00%
Epoch 180/200  loss: 0.0086  train accuracy: 100.00%
Epoch 200/200  loss: 0.0075  train accuracy: 100.00%

Test loss     : 0.0866
Test accuracy : 96.67%

Sample predictions (predicted / actual):
5/5 2/2 8/8 1/1 7/7 2/2 6/6 2/2 6/6 5/5
```

RESULT:

Thus a three-layer neural network was implemented from scratch using NumPy and trained with the backpropagation algorithm using sigmoid and softmax activations with cross-entropy loss. The training loss fell from 1.8152 to 0.0075 over 200 epochs, and the network recognized handwritten digits with a test accuracy of 96.67% (test loss 0.0866).